Введение в Go

Занятие 1

О языке Go

Происхождение

- Создан в Google в 2007 году (Роберт Гриземер, Роб Пайк, Кен Томпсон)
- Анонсирован в ноябре 2009 года
- Open-source проект
- Актуальная версия: 1.26

Философия Go

Радикальная простота

- Всего 25 ключевых слов
- Нет наследования, исключений, перегрузки операторов
- Нет конструкторов/деструкторов
- Нет неявных преобразований типов
- Гарантия обратной совместимости

> Как заметил Роб Пайк: "Сложность мультипликативна — попытка решить проблему усложнением одной части неизбежно порождает сложность в других"

Ключевые особенности

- **Быстрая компиляция** — статическая компиляция
- **Строгая типизация** — ошибки на этапе компиляции
- **Единый стиль** — `gofmt` форматирует код
- **Богатая стандартная библиотека**

Параллелизм как фундамент

Go создавался для эпохи многоядерных процессоров:

- **Горутины** — легковесные потоки (тысячи и миллионы одновременно)
- **Каналы** — безопасное взаимодействие между горутинами
- **Модель CSP** — Communicating Sequential Processes

```
go func() {
    ch <- data  // отправка в канал
}()
value := <-ch  // получение из канала
```

Области применения

- Инфраструктурные проекты
- CLI-утилиты (Docker, Hugo)
- Веб-сервисы и API
- Сетевые серверы (Kubernetes, Istio)
- Базы данных (CockroachDB, etcd)

Первая программа

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

Структура программы

Элемент	Описание
`package main`	Пакет с точкой входа
`import "fmt"`	Импорт пакета форматированного ввода-вывода
`func main()`	Точка входа в программу
`{` на той же строке	Обязательное правило!

Правило фигурных скобок

```go
// ПРАВИЛЬНО
func main() {
    fmt.Println("Hello")
}

// ОШИБКА КОМПИЛЯЦИИ
func main()
{
    fmt.Println("Hello")
}
```

Задание

Время: 10 минут

Создайте файл `hello.go` с программой "Hello, World!". Скомпилируйте и запустите её:

```
go build hello.go
./hello
```

Измените выводимое сообщение на "Привет, Go!" и запустите снова.

Типы данных, переменные, константы

Объявление переменных

```
var x int = 10        // полная форма
var x = 10            // вывод типа
var x int             // нулевое значение (0)

x := 10               // короткое объявление (только в функциях)
x, y := 10, "hello"   // множественное присваивание
```

Групповое объявление

```
var (
    x int
    y = 20
    z int = 30
    a, b = 40, "hello"
)
```

Нулевые значения

В Go нет неинициализированных переменных:

Тип	Нулевое значение
Числа	`0`
bool	`false`
string	`""`
Указатели, срезы, map, каналы	`nil`

Числовые типы

Знаковые: `int8`, `int16`, `int32`, `int64`, `int`

Беззнаковые: `uint8`, `uint16`, `uint32`, `uint64`, `uint`

Специальные: `byte` (= uint8), `rune` (= int32)

С плавающей точкой: `float32`, `float64`

> `int` и `uint` — платформозависимые (32 или 64 бита)

Нет неявных преобразований!

```go
var x int = 10
var y float64 = 30.2

// sum := x + y  // ОШИБКА!

var sum float64 = float64(x) + y  // Правильно
```

Строки и руны

Строки — это последовательность байтов

```
s := "Hello,     "
fmt.Println(len(s))    // 13 байт (не 9 символов!)

r := '€'               // руна (int32)
fmt.Println(r)         // 8364

for i, r := range "Hello,     " {
    fmt.Printf("%d: %c\n", i, r)
}
```

Подстроки и поиск

Взять N символов (не байтов!):

```
s := "Hello,     "

// Неправильно: s[:5] может разрезать руну!
// Правильно: через конвертацию в руны
runes := []rune(s)
substr := string(runes[:5])  // "Hello"

// Количество рун
import "unicode/utf8"
n := utf8.RuneCountInString(s)  // 9
```

Поиск подстроки:

```
import "strings"

idx := strings.Index(s, "    ")    // 7 — позиция в байтах!
```

Конвертация строк и чисел

```go
import "strconv"

// Строка → число
n, err := strconv.Atoi("123")          // 123, error
n, err := strconv.ParseInt("FF", 16, 64)  // 255 (16-ричное)
f, err := strconv.ParseFloat("3.14", 64)  // 3.14

// Число → строка
s := strconv.Itoa(123)                 // "123"
s := strconv.FormatInt(255, 16)        // "ff" (16-ричное)
s := strconv.FormatFloat(3.14, 'f', 2, 64)  // "3.14"

// Ошибка конвертации
n, err := strconv.Atoi("abc")  // n=0, err!=nil
if err != nil {
    fmt.Println("не число!")
}
```

Константы

```
const x = 10            // нетипизированная
const y int = 10        // типизированная

// const z = x + y      // ОШИБКА: вычисляется во время выполнения
```

Правило: константы вычисляются только во время компиляции

iota — генератор констант

```
const (
    Sunday = iota       // 0
    Monday              // 1
    Tuesday             // 2
)

const (
    _ = iota            // 0 пропущен
    KB = 1 << (10 * iota)  // 1024
    MB                     // 1048576
)
```

Ваши вопросы?

Задание

Время: 10 минут

Сколько минут до полудня?

Дана строка вида `"10:34"` — время до 12:00. Может быть с ведущими нулями (`"09:05"`) или без (`"9:5"`).

Напишите программу, которая:

1. Находит позицию двоеточия (`strings.Index`)
2. Берёт две подстроки (до и после двоеточия) по найденному индексу
3. Преобразует части в числа (`strconv.Atoi`)
4. Вычисляет, сколько минут осталось до 12:00

```
// Примеры:
```

Управляющие конструкции

Оператор if

```
if x > 5 {
    fmt.Println(x)
} else if x < 0 {
    fmt.Println("negative")
} else {
    fmt.Println("small")
}
```

Переменная в условии if

```
if n := rand.Intn(10); n == 0 {
    fmt.Println("zero")
} else if n > 5 {
    fmt.Println("big:", n)
} else {
    fmt.Println("small:", n)
}
// fmt.Println(n)  // ОШИБКА: n не определена
```

Цикл for — единственный в Go

```go
// Полная форма
for i := 0; i < 10; i++ {
    fmt.Println(i)
}

// Как while
i := 1
for i < 100 {
    i = i * 2
}

// Бесконечный
for {
    // ...
    if done { break }
}
```

for-range

```go
// Срезы
for i, v := range []int{2, 4, 6} {
    fmt.Println(i, v)
}

// Map (порядок случайный!)
for k, v := range map[string]int{"a": 1} {
    fmt.Println(k, v)
}

// Строка (по рунам!)
for i, r := range "Hello,    " {
    fmt.Printf("%d: %c\n", i, r)
}
```

break и continue

```
for i := 0; i < 10; i++ {
    if i == 5 { break }       // выход из цикла
    if i%2 == 0 { continue } // следующая итерация
    fmt.Println(i)
}
```

Метки для вложенных циклов

```
outer:
for _, sample := range samples {
    for i, r := range sample {
        if r == 'l' {
            continue outer    // к следующей итерации внешнего цикла
        }
        fmt.Println(i, r)
    }
}
```

Оператор switch

```go
switch size := len(word); size {
case 1, 2, 3, 4:
    fmt.Println("short")
case 5:
    fmt.Println("medium")
default:
    fmt.Println("long")
}
```

Нет fallthrough по умолчанию!

Пустой switch

```
switch {
case score >= 90:
    fmt.Println("A")
case score >= 80:
    fmt.Println("B")
case score >= 70:
    fmt.Println("C")
default:
    fmt.Println("F")
}
```

Затенение переменных

```
x := 10
if x > 5 {
    x := 5    // НОВАЯ переменная! Затенение!
    fmt.Println(x)  // 5
}
fmt.Println(x)  // 10 (внешняя)
```

Внимание: `:=` может случайно создать затенение!

Ваши вопросы?

Задание

Время: 10 минут

Проверка сложности пароля

Напишите программу, которая проверяет пароль на сложность:

1. Объявите строку `password := "MyPass123"`
2. Пройдитесь по строке циклом `for-range`
3. Подсчитайте: количество заглавных букв, строчных букв и цифр
4. Используйте `if` с объявлением переменной для проверки результатов:

```
if hasUpper := upperCount > 0; hasUpper && lowerCount > 0 && digitCount > 0 {
    fmt.Println("Пароль надёжный")
} else {
    fmt.Println("Пароль слабый")
}
```

Сегодня мы узнали

- **О языке** — 25 ключевых слов, простота, параллелизм
- **Первая программа** — package main, import, правило `{`
- **Переменные** — var, :=, нулевые значения, типы
- **Константы** — const, iota, нетипизированные
- **Управляющие конструкции** — if, for, switch, break/continue
- **Затенение** — осторожно с `:=` во вложенных блоках